Name-Vineet Gajwani

Batch-T12, Roll no-2401028

Subject-ASD&D

Experiment 8: CI/CD PIPELINE TO HOST A WEBSITE ON AWS

Aim:

To build an automated CI/CD pipeline on AWS that fetches source code from GitHub, builds it with AWS CodeBuild, and deploys it to an Amazon S3 static website using AWS CodePipeline.

This experiment maps to LO4 & LO6.

LO4 : Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

LO6 : Develop and demonstrate an end-to-end DevOps pipeline mini-project integrating all stages of DevOps from code commit to deployment and monitoring.

Theory:

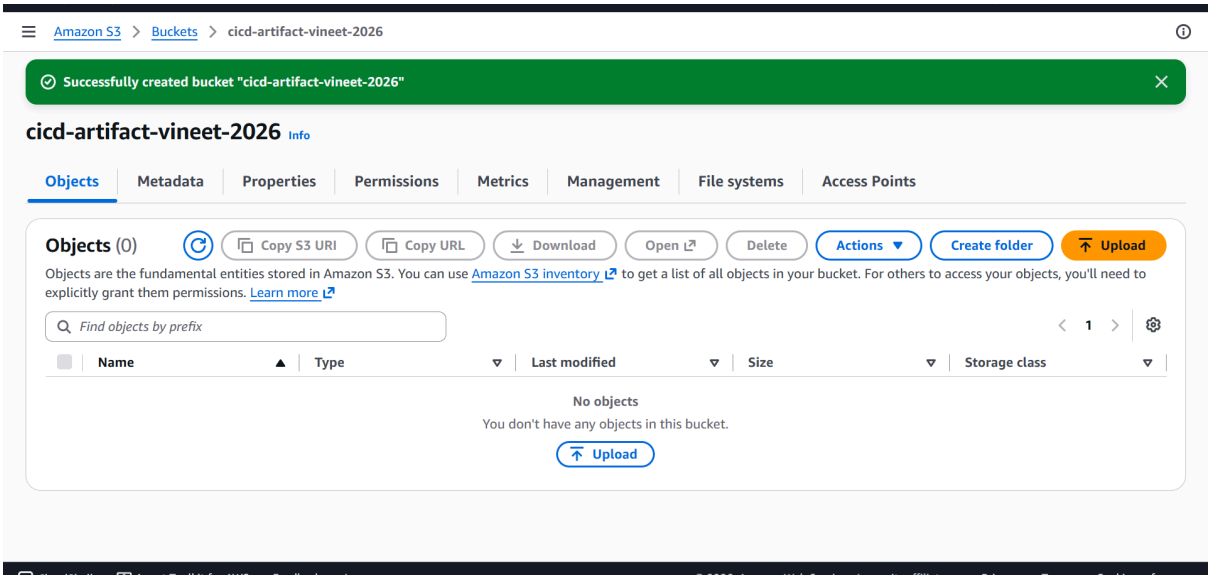
Continuous Integration and Continuous Delivery (CI/CD) is a DevOps practice in which code changes committed to a source repository are automatically built, tested, and prepared for release through a defined sequence of pipeline stages. Continuous Integration ensures that each commit is merged and verified by an automated build system so integration problems surface early. Continuous Delivery and Deployment then carry the verified build artifacts through to target environments without requiring manual intervention. This approach significantly shortens the feedback loop between writing code and deploying it, making software releases small, frequent, and repeatable.

AWS offers a suite of fully managed developer tools for implementing automated CI/CD workflows. AWS CodePipeline serves as the central orchestrator, modeling release workflows into sequential stages—such as Source, Build, Test, and Deploy—and transferring output artifacts stored in Amazon S3 between stages. AWS CodeBuild is a serverless build service that spins up temporary build environments, pulls source code, and executes the specified build commands. Amazon S3 functions both as an artifact repository for intermediate build outputs and as a web hosting platform when configured with public read access. Connected via webhooks to source repositories like GitHub, the pipeline automatically triggers on code pushes, ensuring the live application continuously reflects the latest commits.

Implementation:

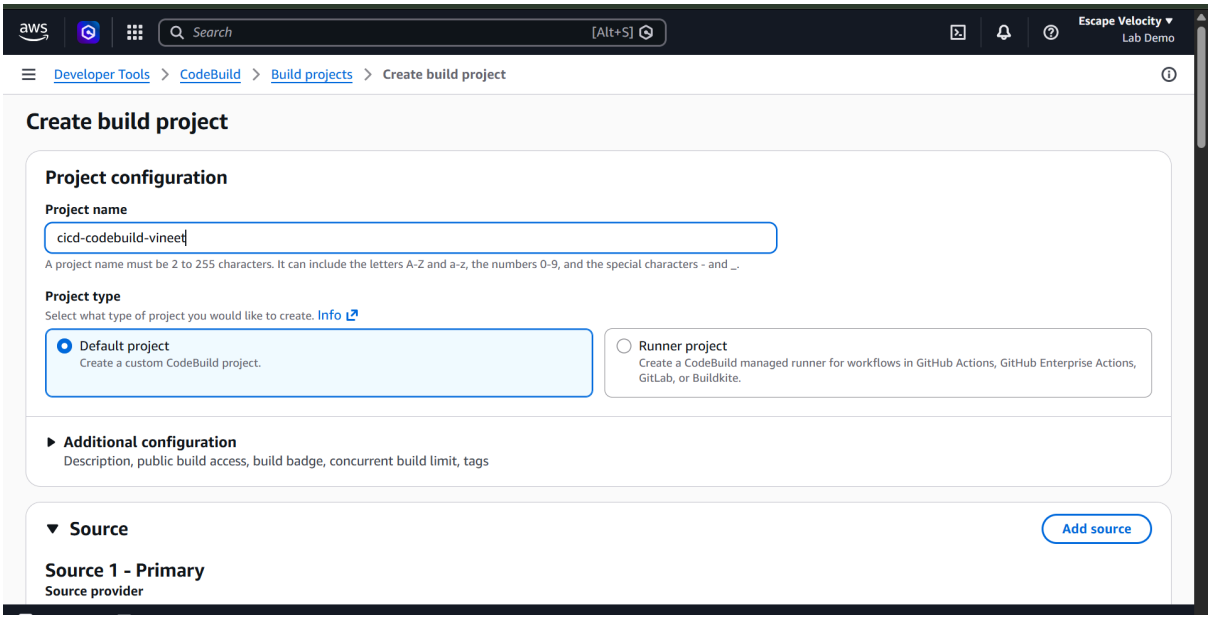
1. Create the Artifact Bucket:

An S3 bucket named cicd-artifact-vineet-2026 is created to act as the artifact store for the pipeline. CodePipeline uses this bucket to hold the source and build artifacts that are passed between stages, so it is created empty before the pipeline itself is configured.



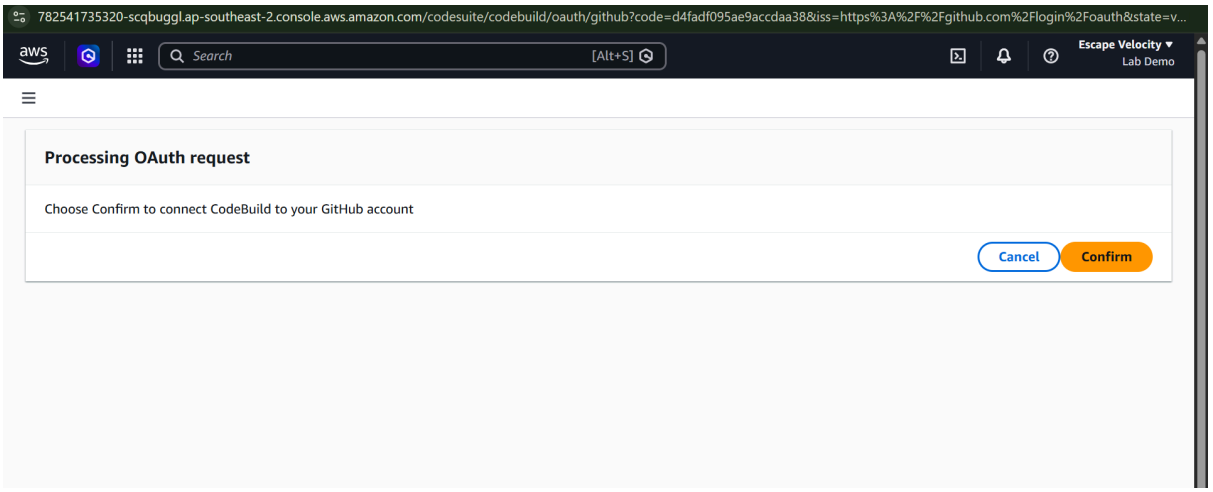
2. Create the CodeBuild Project:

In the CodeBuild console a new build project named cicd-codebuild-vineet is created with the Default project type. This project defines the build environment and the steps that CodeBuild executes whenever the pipeline reaches the Build stage.



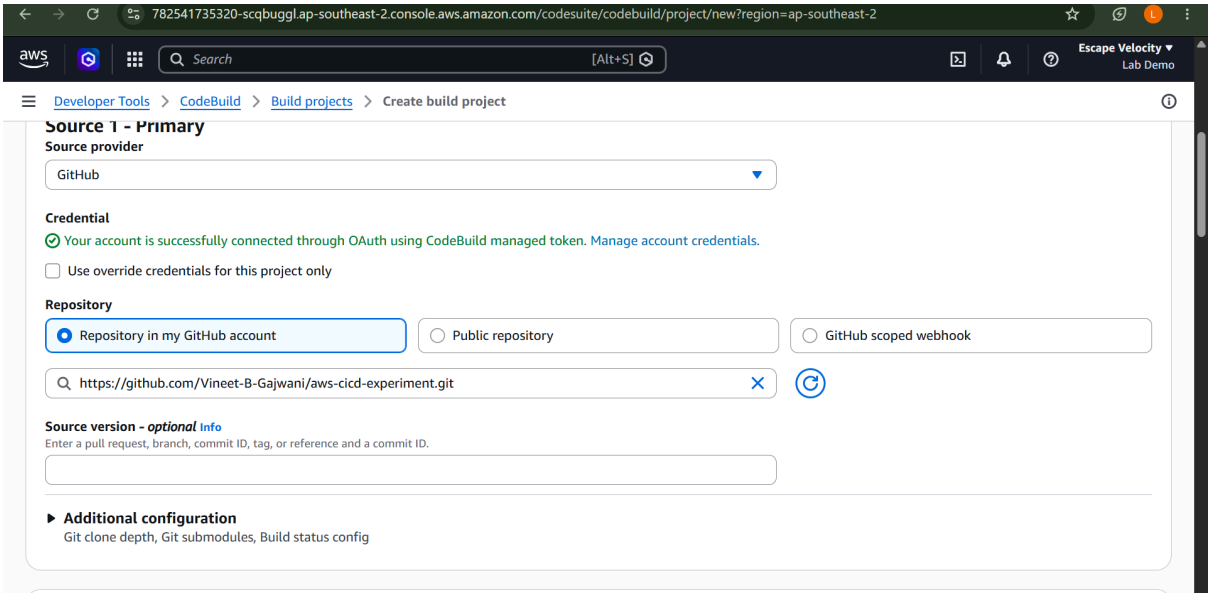
3. Connect CodeBuild to GitHub:

GitHub is selected as the source provider and CodeBuild is authorised through an OAuth request. Choosing Confirm links the GitHub account to CodeBuild using a CodeBuild-managed token, allowing it to read the repositories owned by the account.



4. Select the Source Repository:

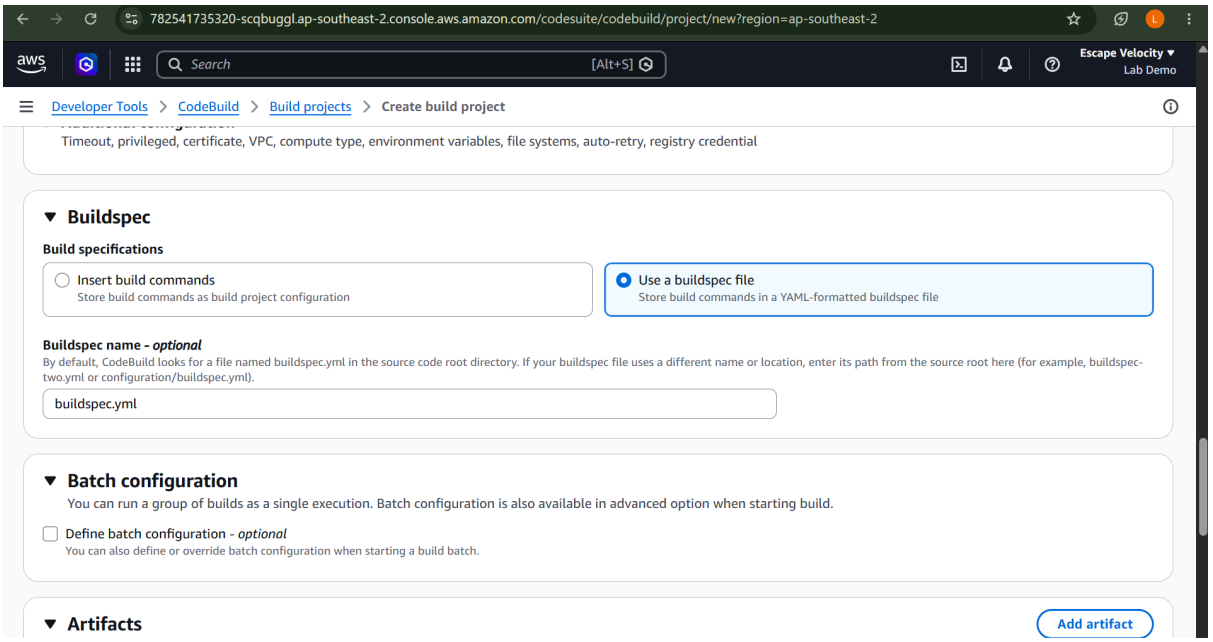
With the OAuth connection established, the option Repository in my GitHub account is chosen and the repository aws-cicd-experiment is selected as the primary source. The source version is left blank so that the default branch is built.



5. Configure the Buildspec:

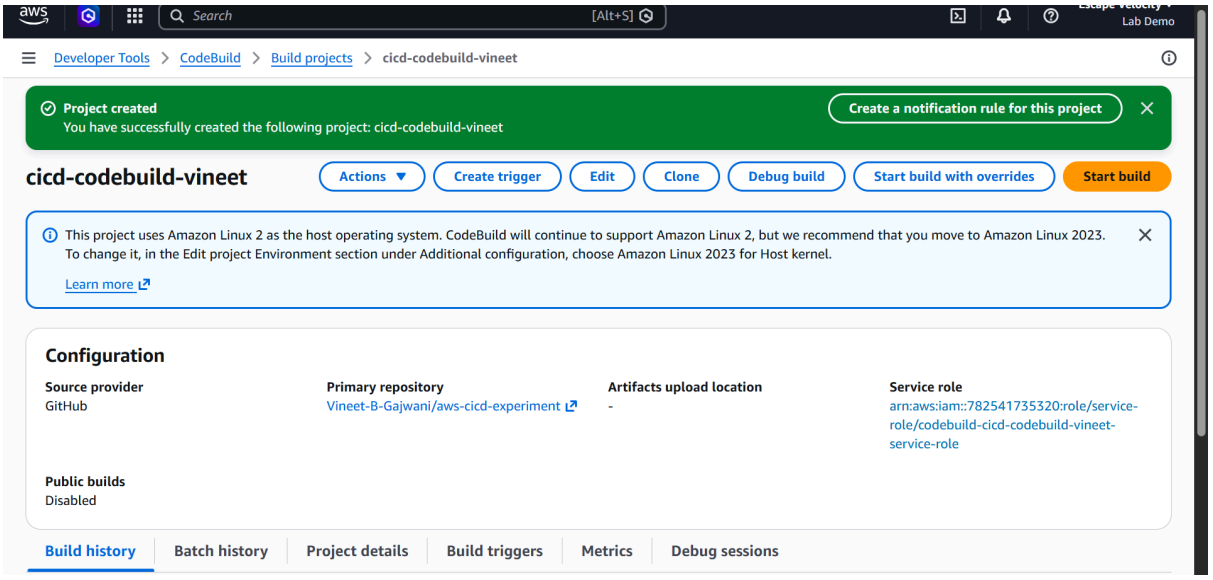
Under Build specifications the option Use a buildspec file is selected, with the file name buildspec.yml. CodeBuild therefore reads its install, build, and artifact instructions from the

YAML file committed in the root of the repository instead of storing commands in the project configuration.



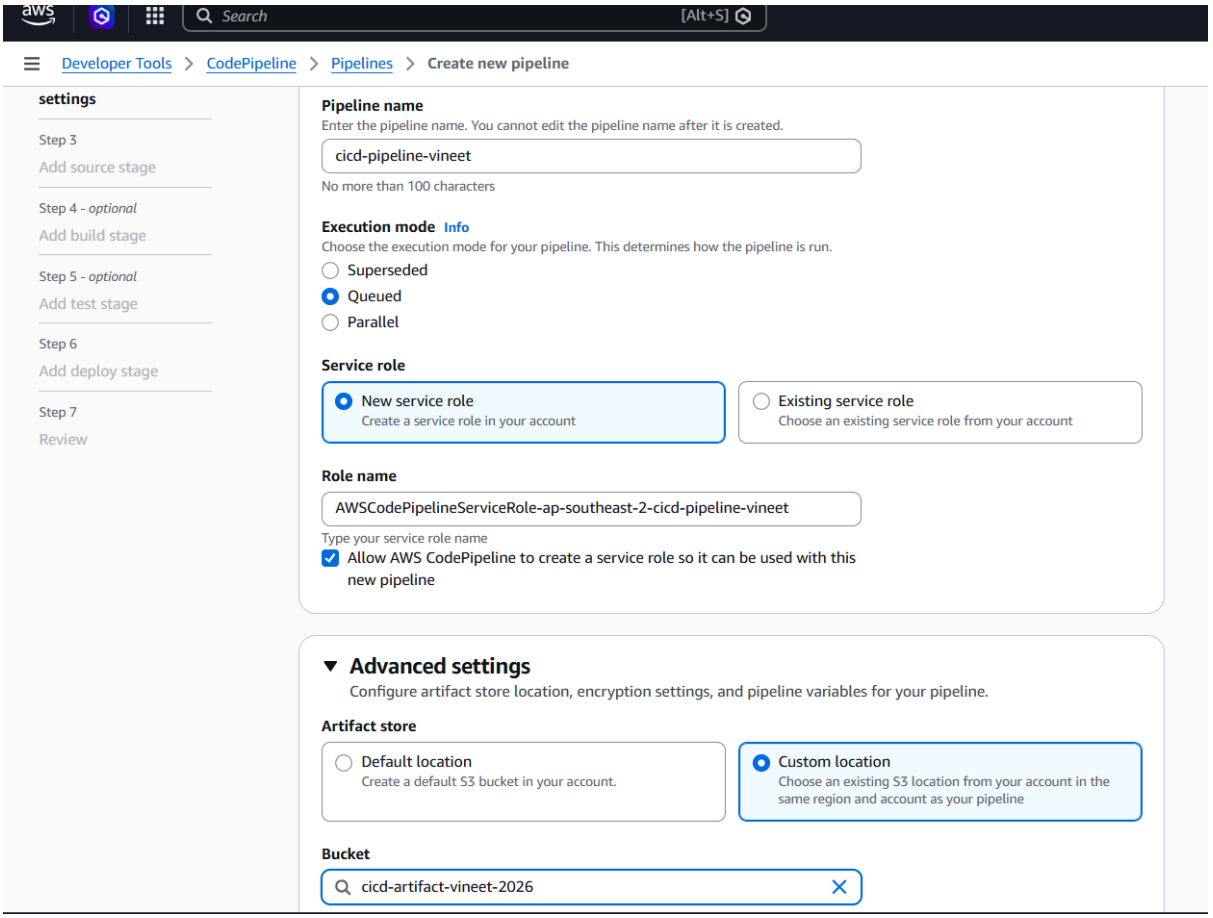
6. Verify the Build Project:

The project is created successfully. The configuration panel confirms GitHub as the source provider, Vineet-B-Gajwani/aws-cicd-experiment as the primary repository, and an automatically generated IAM service role that grants CodeBuild the permissions it needs during a build.



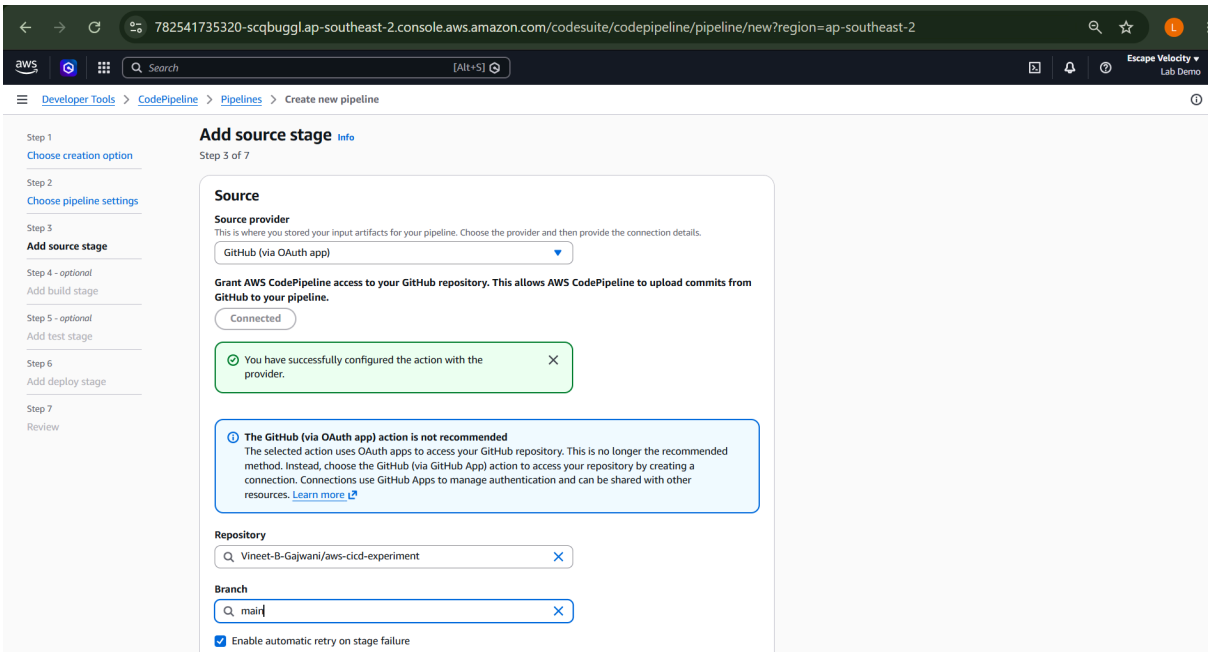
7. Configure the Pipeline Settings:

In CodePipeline a new pipeline named cicd-pipeline-vineet is created in Queued execution mode, together with a new service role. Under Advanced settings a custom artifact store is chosen and pointed at the cicd-artifact-vineet-2026 bucket created in the first step.



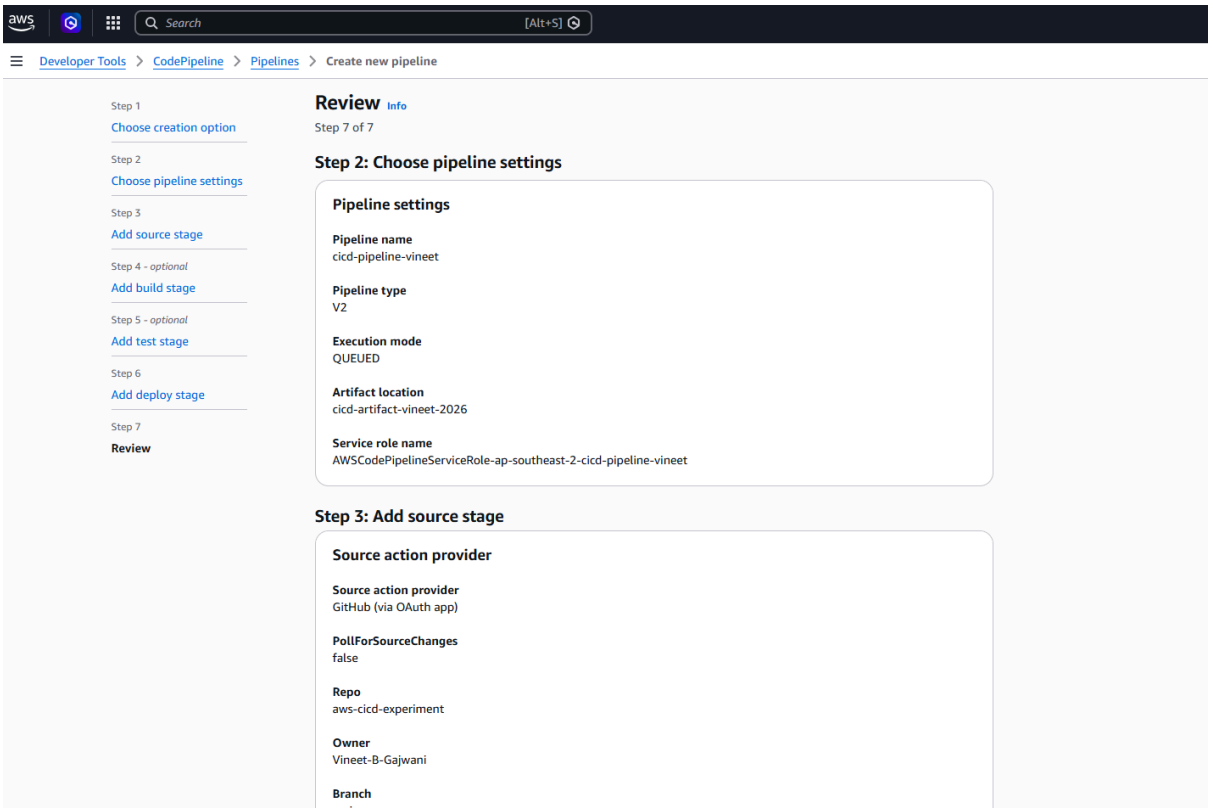
8. Add the Source Stage:

GitHub (via OAuth app) is chosen as the source provider and the connection is confirmed. The repository Vineet-B-Gajwani/aws-cicd-experiment and the branch main are selected, so that every commit to main becomes a new input to the pipeline.



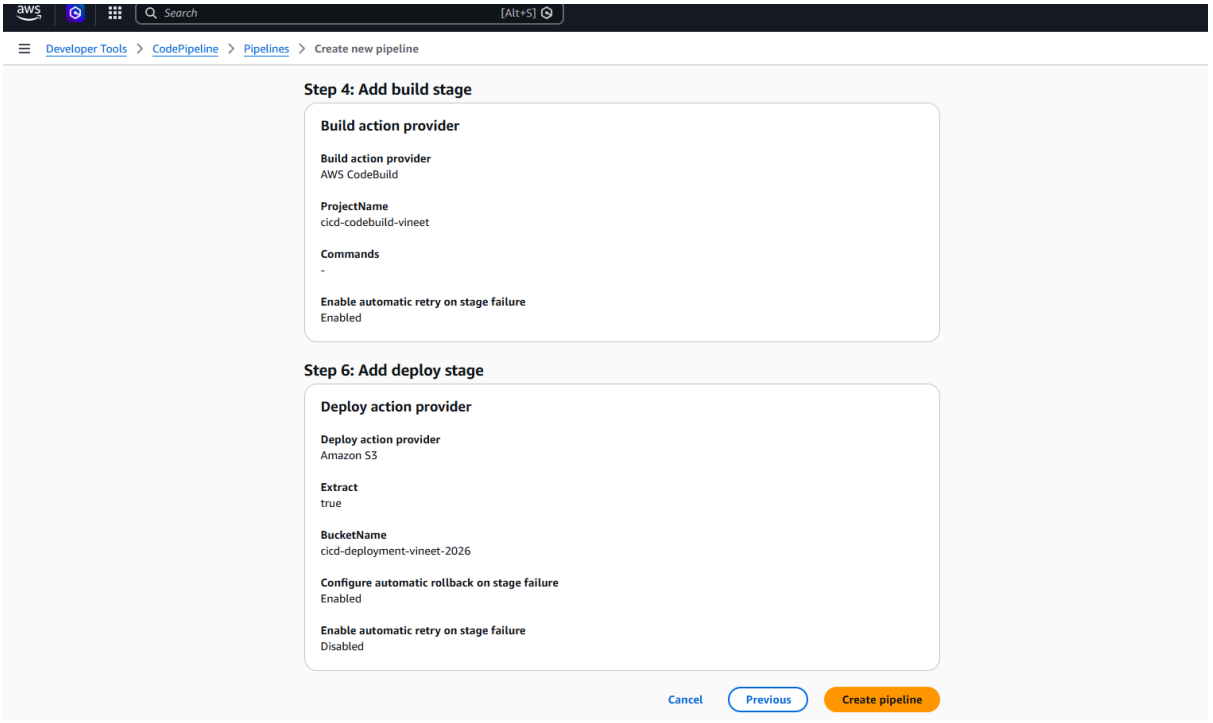
9. Review the Pipeline and Source Settings:

The review page summarises the pipeline configuration: pipeline type V2, execution mode QUEUED, the custom artifact location, and the generated service role. The source stage lists GitHub as the provider along with the repository owner, name, and branch.



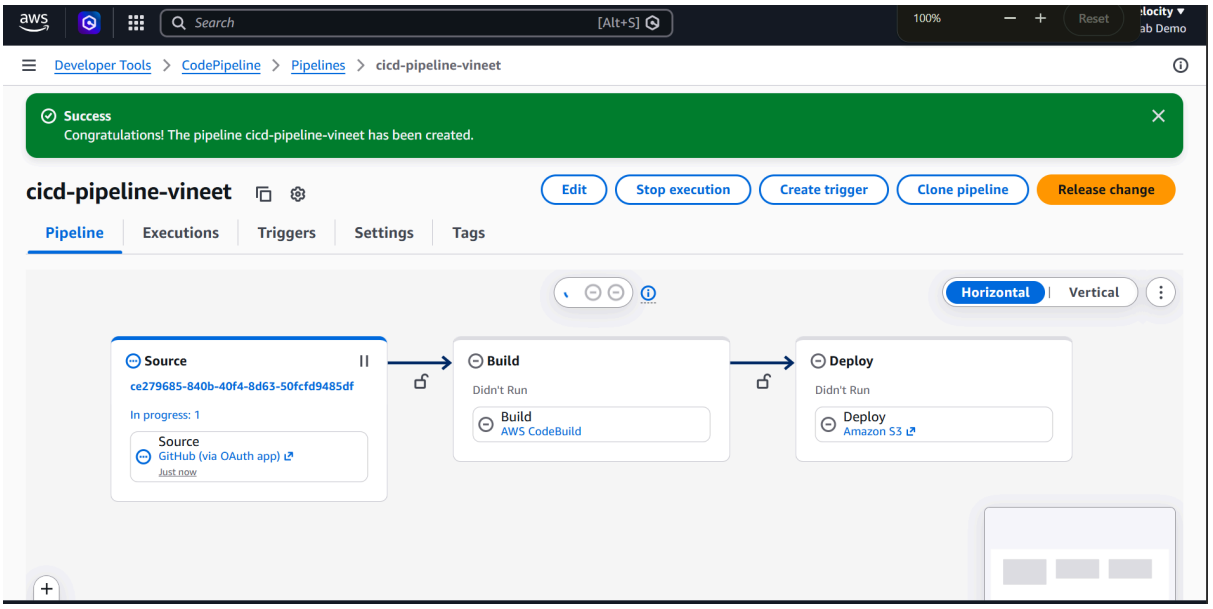
10. Review the Build and Deploy Stages:

The build stage uses AWS CodeBuild with the project cicd-codebuild-vineet, and the deploy stage uses Amazon S3 with the bucket cicd-deployment-vineet-2026. Extract is set to true so that the build output is unzipped into the bucket as individual website files. Choosing Create pipeline saves the configuration.



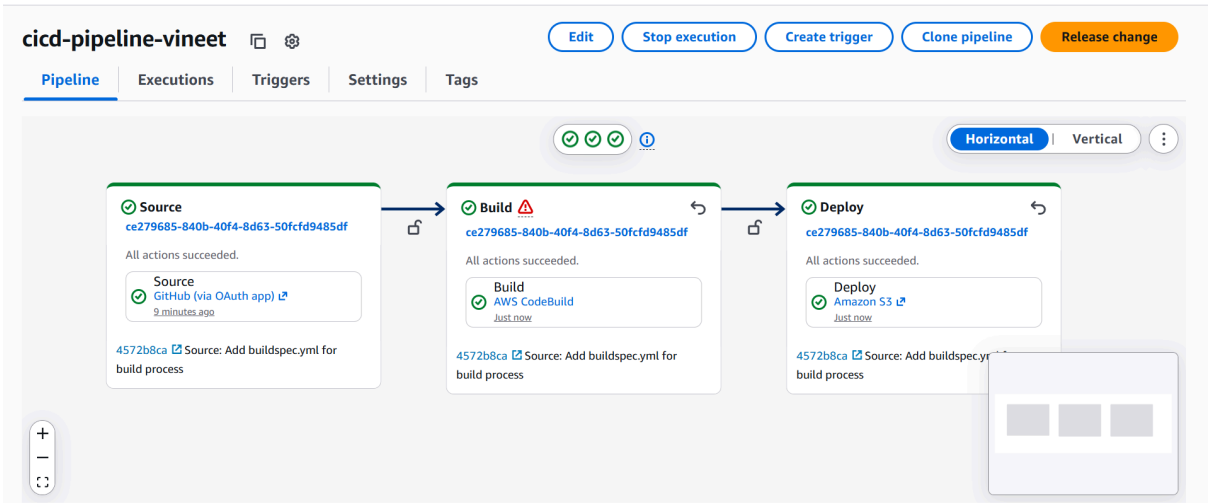
11. Run the Pipeline for the First Time:

The pipeline is created and its first execution starts immediately. The Source stage is in progress fetching the latest commit, while the Build and Deploy stages are waiting for their input artifacts.



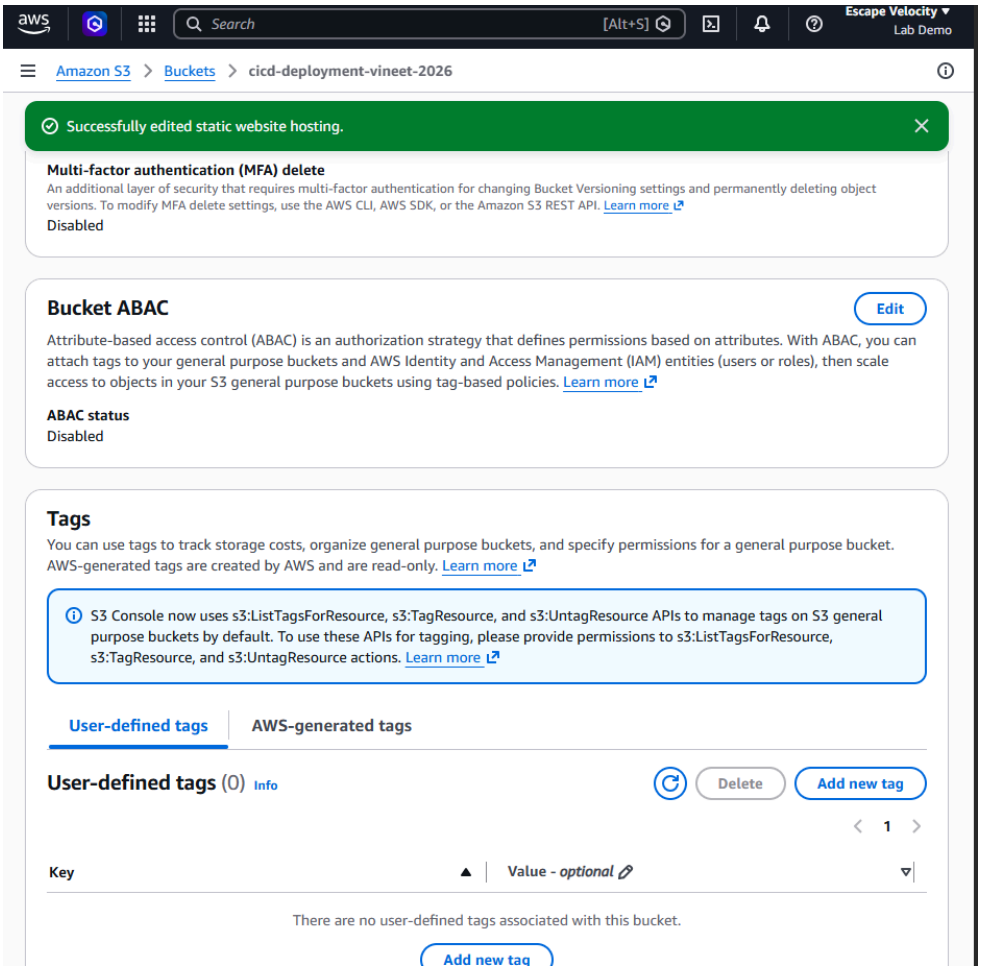
12. Verify a Successful Execution:

All three stages — Source, Build, and Deploy — complete successfully for the commit 4572b8ca, “Add buildspec.yml for build process”. The same commit identifier appears in every stage, showing that a single revision flowed through the entire pipeline.



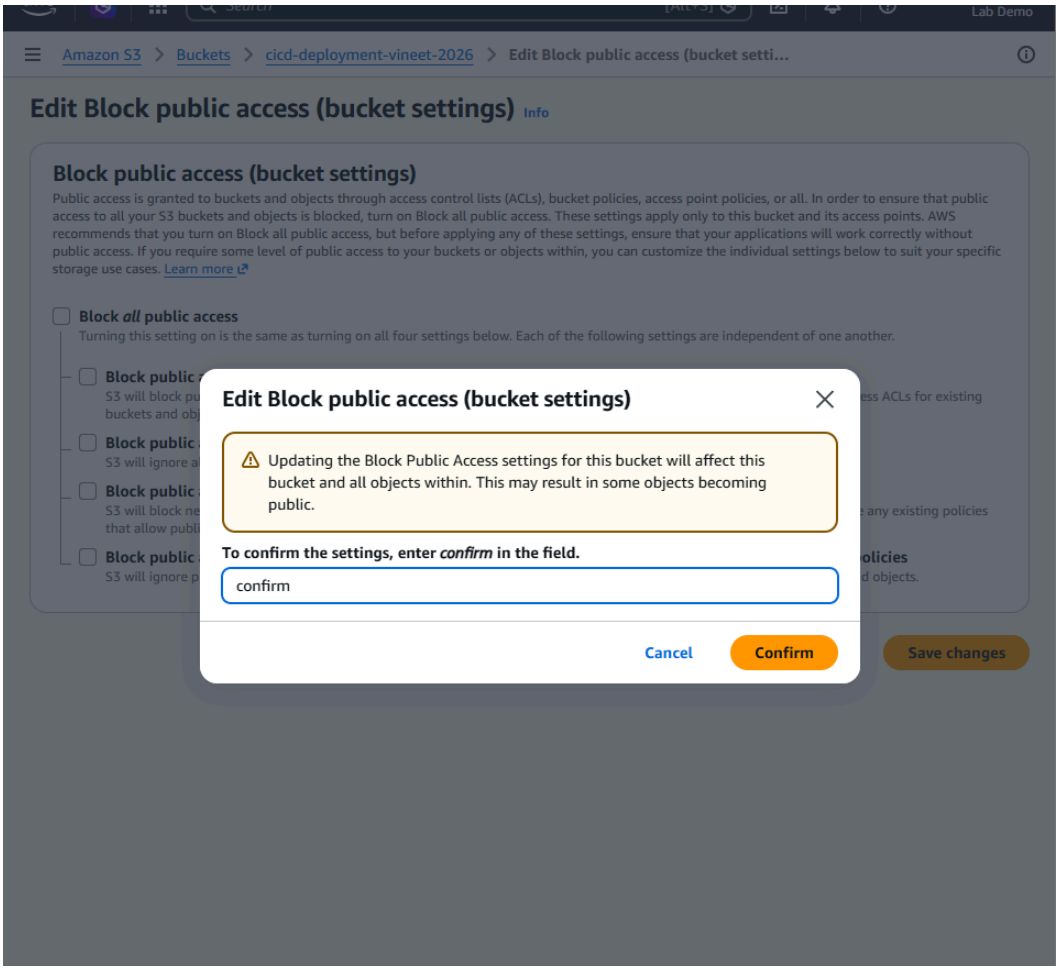
13. Enable Static Website Hosting:

In the properties of the deployment bucket cicd-deployment-vineet-2026, static website hosting is enabled so that S3 serves index.html as the default document through a public website endpoint.



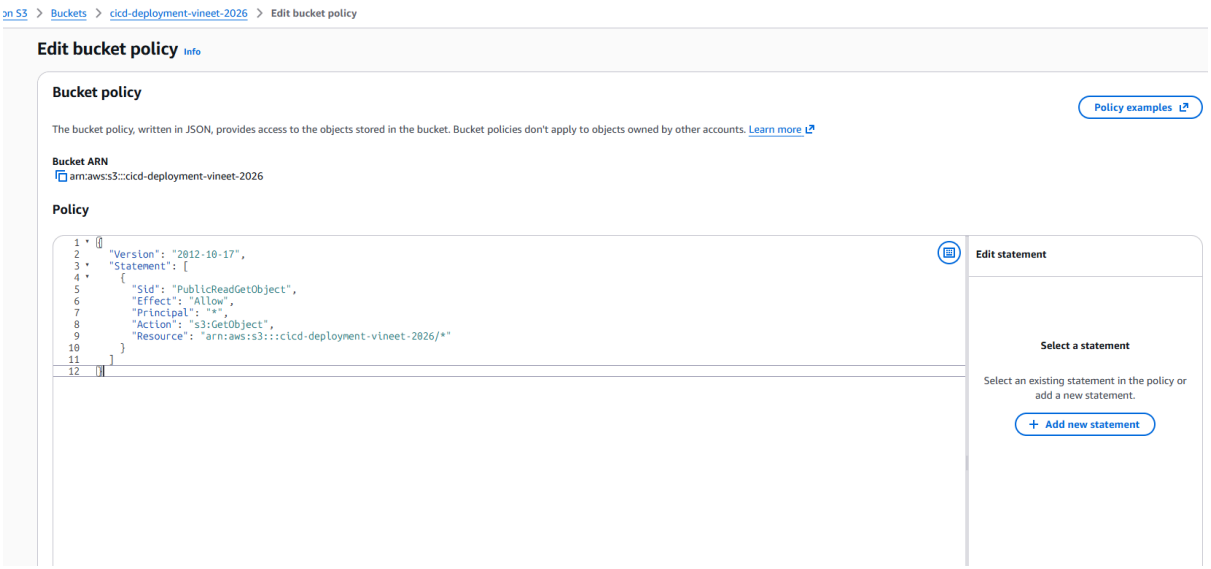
14. Disable Block Public Access:

S3 blocks all public access to new buckets by default. To let visitors load the website, Block all public access is turned off for the deployment bucket, and the change is acknowledged by typing confirm.



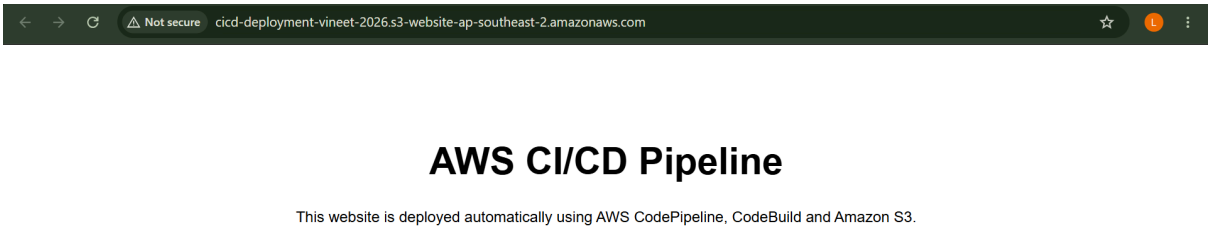
15. Attach a Bucket Policy:

A bucket policy is added that allows the s3:GetObject action for every principal on all objects in the bucket. This grants anonymous read-only access to the website files without allowing any uploads or deletions.



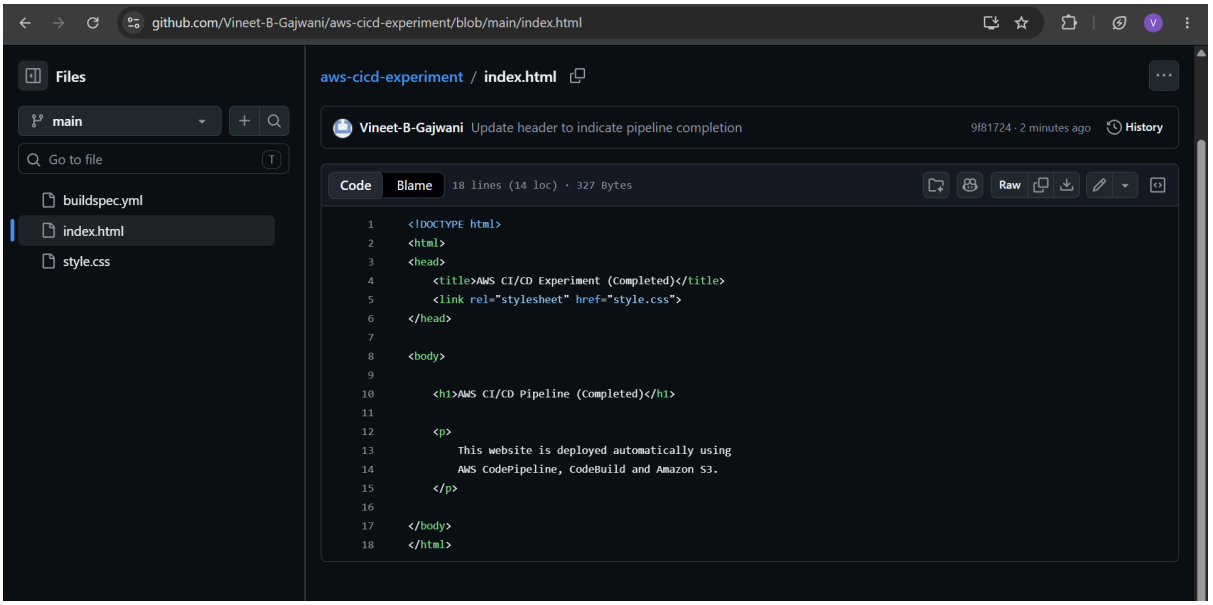
16. Access the Deployed Website:

Opening the S3 website endpoint in a browser displays the page titled AWS CI/CD Pipeline, confirming that the files built by CodeBuild were successfully deployed to S3 by the pipeline.



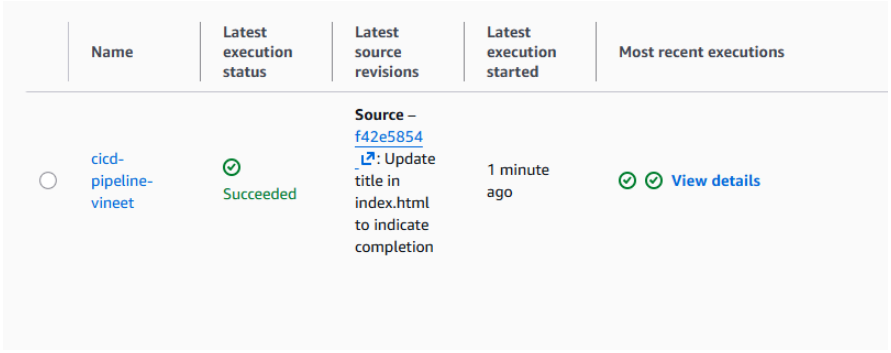
17. Push an Update to the Repository:

To test continuous deployment, index.html is edited on GitHub so that the heading reads AWS CI/CD Pipeline (Completed), and the change is committed directly to the main branch.



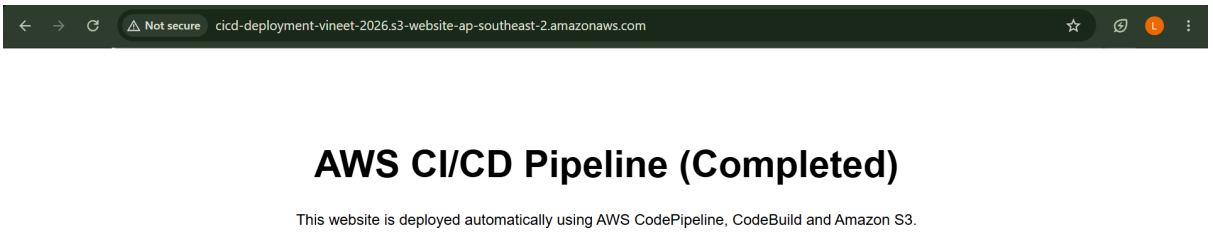
18. Observe the Automatic Pipeline Trigger:

Without any manual action, the pipeline detects the new commit and starts a fresh execution. The pipeline list shows the latest execution status as Succeeded along with the commit message of the update.



19. Verify the Website After the Update:

Refreshing the website endpoint now shows the updated heading AWS CI/CD Pipeline (Completed). The change travelled from a GitHub commit to the live site entirely through the automated pipeline.



Conclusion:

In this experiment, an automated CI/CD pipeline was successfully implemented on AWS using CodePipeline, CodeBuild, and Amazon S3. The process began with creating an S3 artifact bucket, followed by configuring a CodeBuild project linked to a GitHub repository via OAuth, using buildspec.yml to define the build steps. A V2 pipeline was configured with three sequential stages: a GitHub source stage tracking the main branch, a CodeBuild build stage, and an Amazon S3 deploy stage. Upon creation, the initial pipeline execution completed successfully, seamlessly passing the commit artifact through every stage.

To serve the application, the deployment bucket was converted into a public static website by enabling static website hosting, disabling Block Public Access, and applying a read-only bucket policy. This rendered the application accessible via the S3 website endpoint. Finally, committing an update to index.html on GitHub automatically triggered the pipeline, instantly updating the live site without manual intervention. This demonstrated the core benefit of continuous delivery: automated, reliable, and repeatable deployments for every code change.